Modern Regression

Elastic Net and Bayesian Sparse Regression

Jyotishka Datta

2023-03-31

Elastic Net

Two independent hidden factors z_1 and z_2:

```
n = 100
z_1 = runif(n,0,20); z_2 = runif(n,0,20)
```

Generate the response as $y = z_1 + 0.1 * z_2 + N(0, 1)$

```
y = z_1 + 0.1*z_2 + rnorm(n)
```

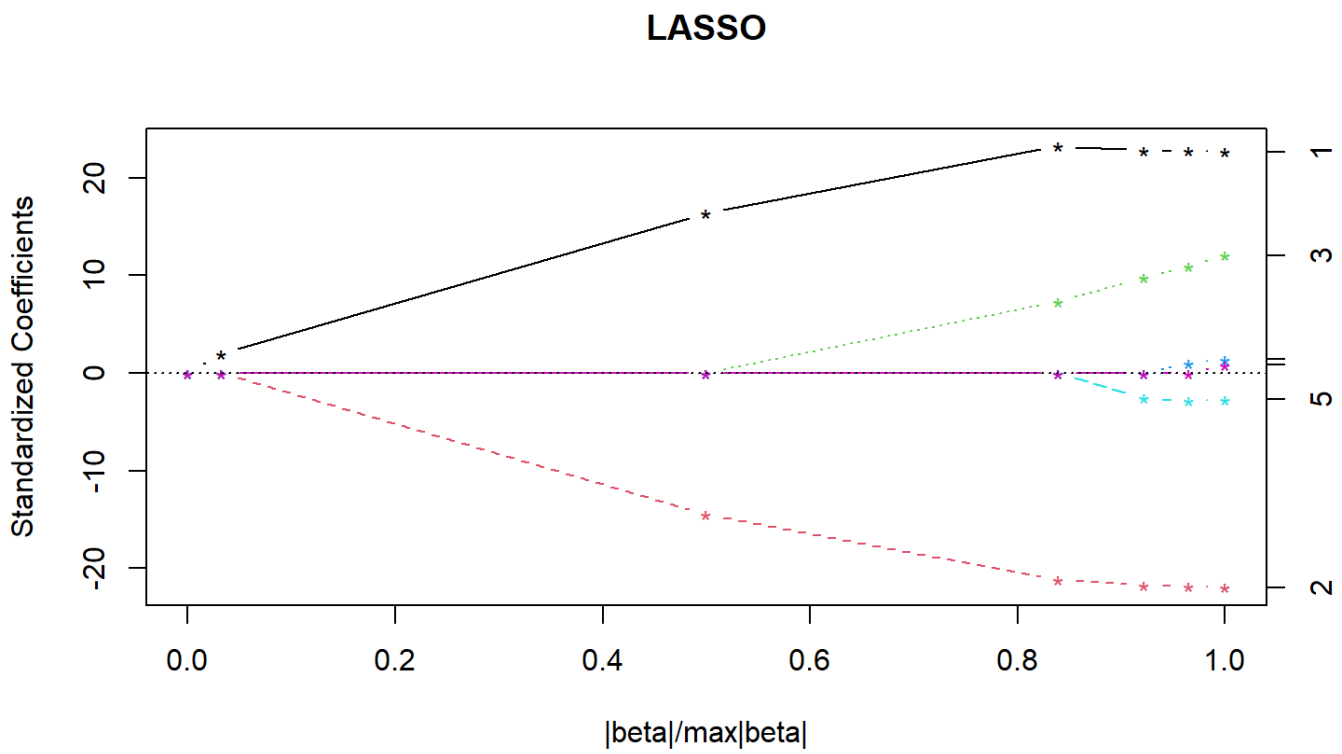
Observe predictors:

$$x_1 = z_1 + \epsilon_1, x_2 = -z_1 + \epsilon_2, x_3 = z_1 + \epsilon_3$$
$$x_4 = z_2 + \epsilon_4, x_5 = -z_2 + \epsilon_5, x_6 = z_2 + \epsilon_6$$

```
x_1 = z_1 + rnorm(n); x_2 = -z_1 + rnorm(n); x_3 = z_1 + rnorm(n)
x_4 = z_2 + rnorm(n);x_5 = -z_2 + rnorm(n);x_6 = z_2 + rnorm(n)
x = cbind(x_1,x_2,x_3,x_4,x_5,x_6)
```

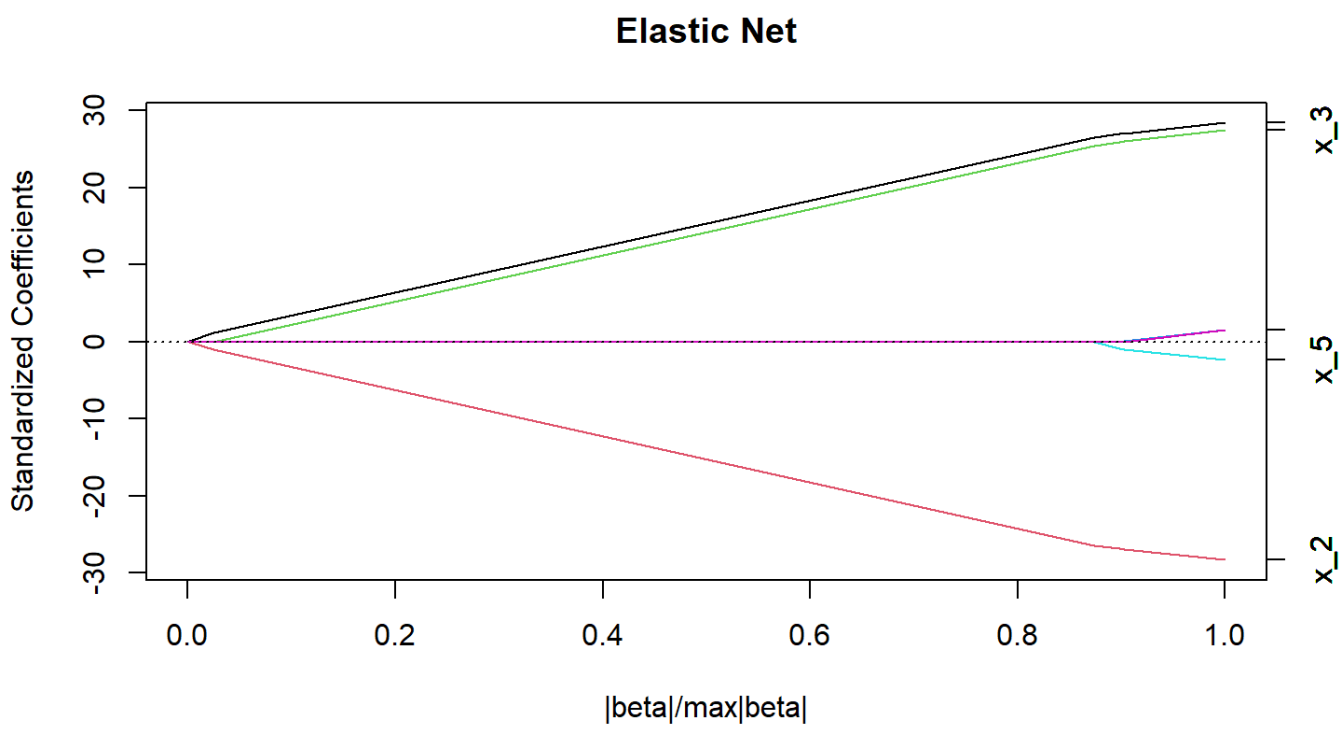
Run Lasso

```
library(lars)
lasso.fit <- lars(x=x,y=y)
plot(lasso.fit, breaks = F)
```



Run Elastic Net

```
if(!require(elasticnet)){
  install.packages('elasticnet',dependencies=TRUE, repos='http://cra
}
library(elasticnet)
en.fit <- enet(x=x,y=y, lambda = 1)
plot(en.fit, use.color = T, main = "Elastic Net")
```



Bayesian Regularization

Bayesian Lasso

- We will look at the same simulation experiment with $p = 60 > n = 30$ where the parameter vector β is sparse.
- We ask: Does Bayesian Lasso produce the same solution as Lasso?

Simulation Experiments

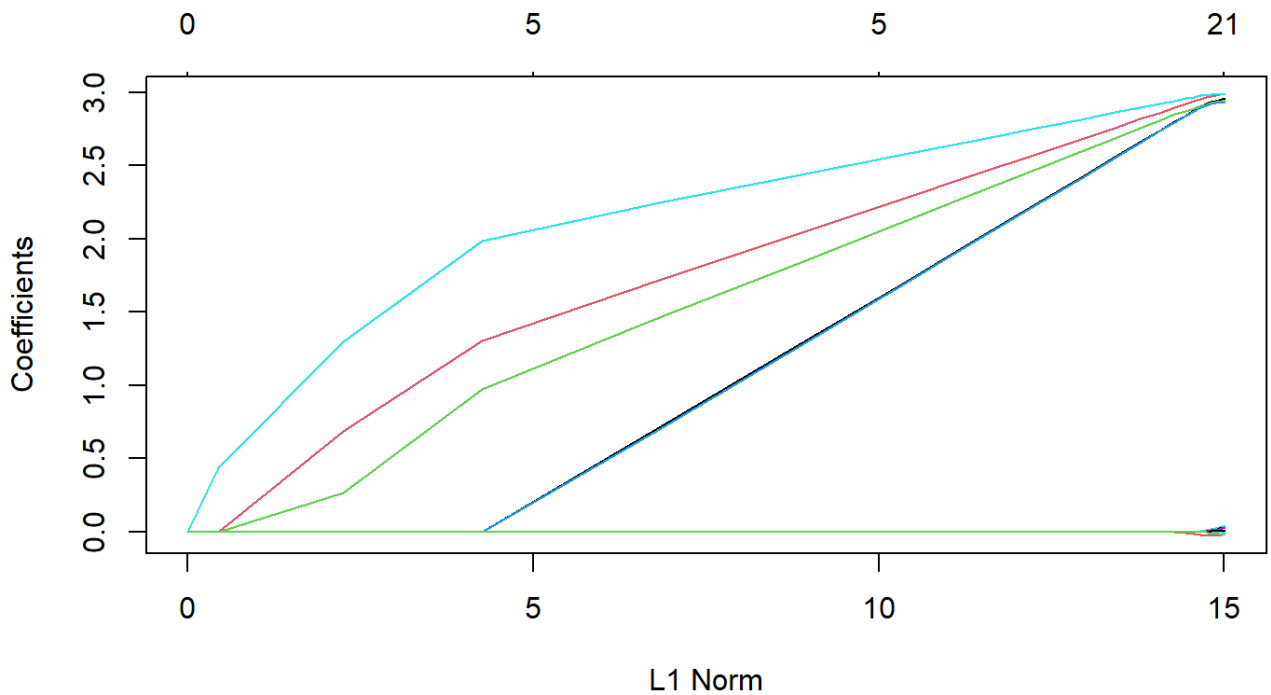
- Generate X, y with $n = 30, p = 60$ with a sparse β vector with 5 non-zero elements.

```
library(Matrix)
set.seed(12345)
n=30;p=60
p1 = 5
beta <- c(rep(3,p1),rep(0,p-p1))
x=scale(matrix(rnorm(n*p),n,p))
eps=rnorm(n,mean=0,sd=0.1)
fx = x %*% beta
y=drop(fx+eps)
```

- Our goal is to see if Lasso can recover the 5 non-zero β's correctly.

Lasso is sparse

```
library(glmnet)
grid=10^seq(10,-2,length=100)
lasso.mod =glmnet(x,y,alpha =1,lambda =grid)
plot(lasso.mod)
```

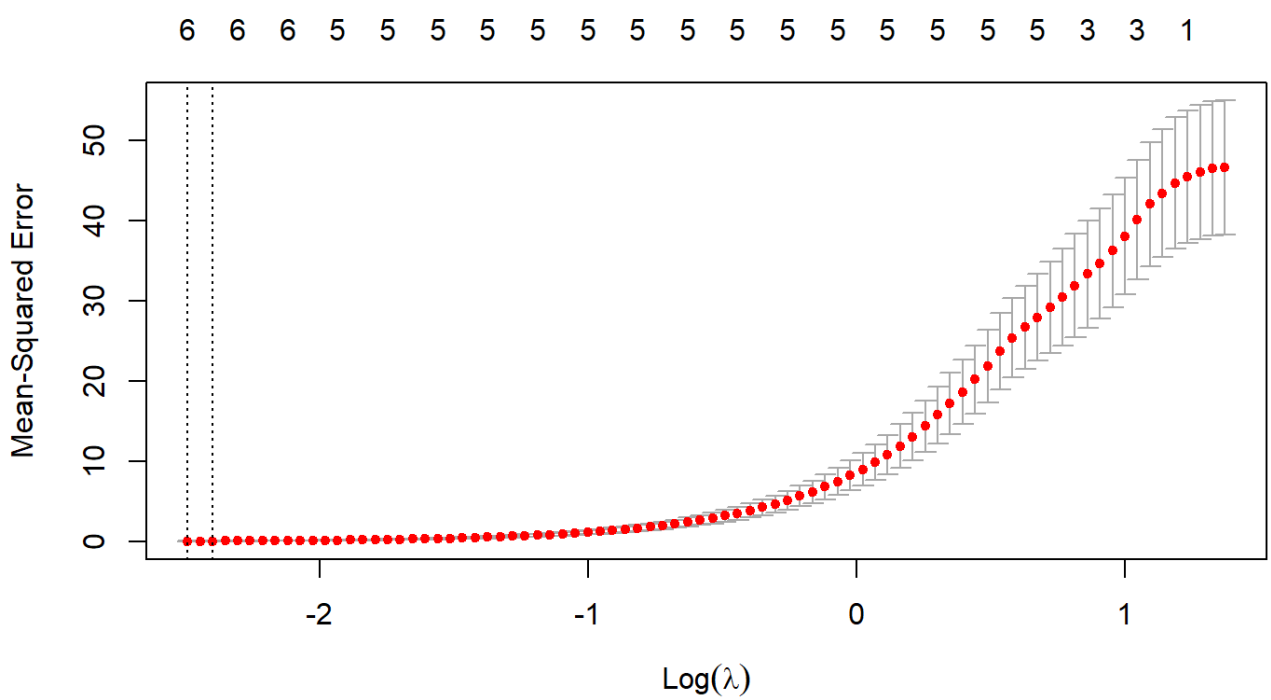


Choosing Lambda

```
cv.out =cv.glmnet(x,y,alpha =1)
(bestlam =cv.out$lambda.min)
```

```
## [1] 0.08295634
```

```
plot(cv.out)
```



Sparse Solution

```
lasso.coef=predict(lasso.mod,type ="coefficients",s=bestlam)
lasso.coef[lasso.coef !=0]
```

```
## [1]  0.01459650  2.85670247  2.92947327  2.88166521  2.85101983
## [7] -0.01440246
```

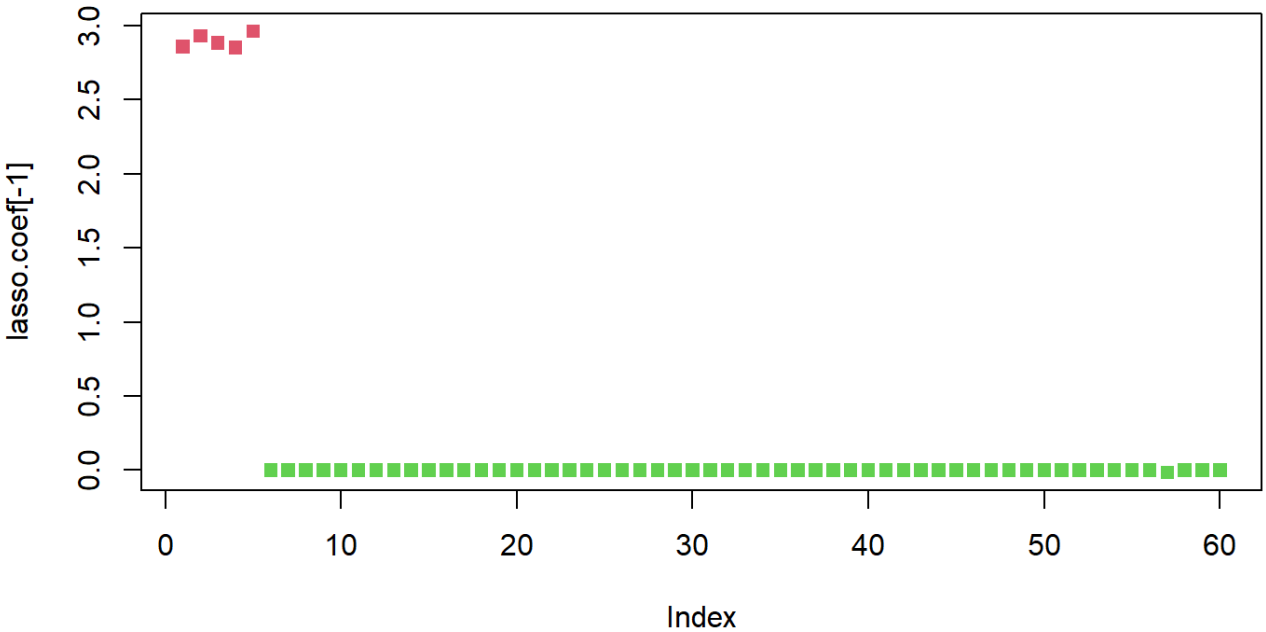
- Use `which` function to get rid of the warning message

```
lasso.coef[which(lasso.coef!=0)]
```

```
## [1]  0.01459650  2.85670247  2.92947327  2.88166521  2.85101983
## [7] -0.01440246
```

LASSO correctly recovers the signals

```
col.index = c(rep(2,p1),rep(3,p-p1))
plot(lasso.coef[-1], col = col.index, pch = 15)
```



Bayesian Lasso

```
library(monomvn)
?blasso
```

Description

Inference for ordinary least squares, **lasso**/NG, horseshoe and **ridge** regression models by (Gibbs) sampling from the Bayesian posterior distribution, augmented with Reversible Jump for model selection.

```
blasso(X, y, T = 1000, thin = NULL, RJ = TRUE, M = NULL,
       beta = NULL, lambda2 = 1, s2 = var(y-mean(y)),
       case = c("default", "ridge", "hs", "ng"), mprior = 0, rd = NU
       ab = NULL, theta = 0, rao.s2 = TRUE, icept = TRUE,
       normalize = TRUE, verb = 1)
```

Help

Description

Inference for ordinary least squares, lasso/NG, horseshoe and ridge regression models by (Gibbs) sampling from the Bayesian posterior distribution, augmented with Reversible Jump for model selection

Usage

```
bhs(X, y, T=1000, thin=NULL, RJ=TRUE, M=NULL, beta=NULL,
         lambda2=1, s2=var(y-mean(y)), mprior=0, ab=NULL,
         theta=0, rao.s2=TRUE, icept=TRUE, normalize=TRUE, verb=
1)
bridge(X, y, T = 1000, thin = NULL, RJ = TRUE, M = NULL,
       beta = NULL, lambda2 = 1, s2 = var(y-mean(y)), mprior = 0,
       rd = NULL, ab = NULL, theta=0, rao.s2 = TRUE, icept = TRU
E,
       normalize = TRUE, verb = 1)
blasso(X, y, T = 1000, thin = NULL, RJ = TRUE, M = NULL,
       beta = NULL, lambda2 = 1, s2 = var(y-mean(y)),
       case = c("default", "ridge", "hs", "ng"), mprior = 0, rd =
NULL,
       ab = NULL, theta = 0, rao.s2 = TRUE, icept = TRUE,
       normalize = TRUE, verb = 1)
```

Arguments

`X`	`data.frame`, `matrix`, or vector of inputs `X`
`y`	vector of output responses `y` of length equal to the leading dimension (rows) of `X`, i.e., `length(y) == nrow(X)`
`T`	total number of MCMC samples to be collected

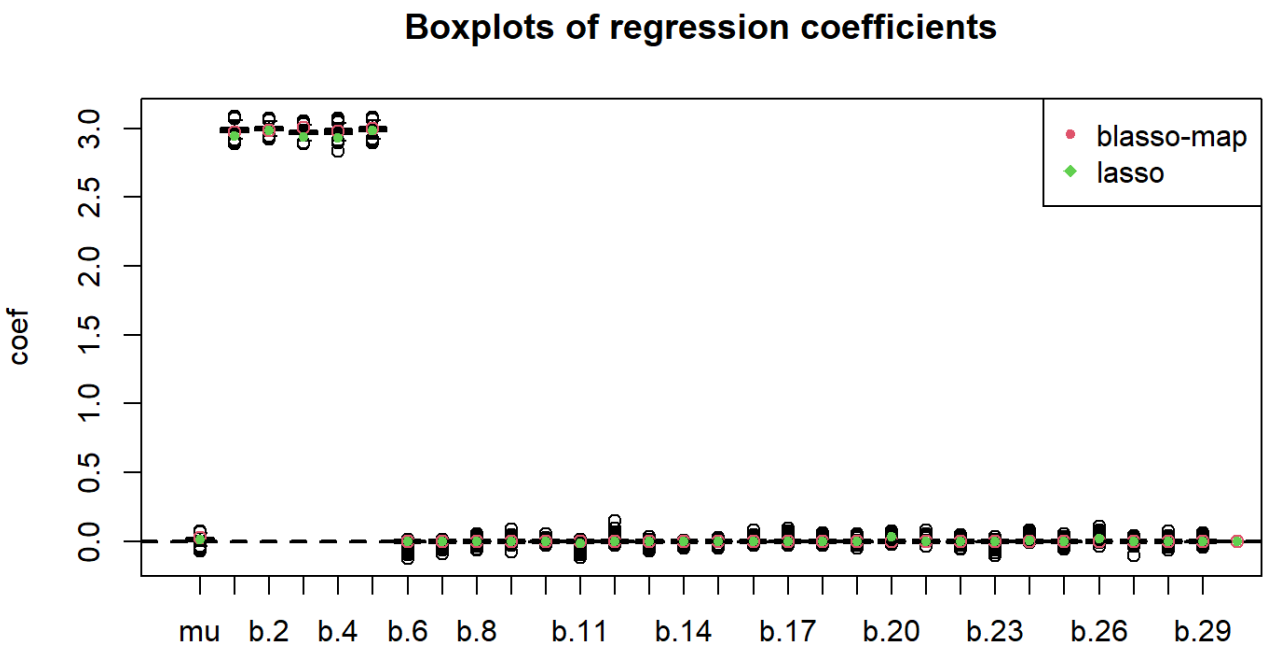
Apply Bayesian Lasso to sparse data

```
library(monomvn)
## Lasso regression
reg.las <- regress(x, y, method="lasso")
## Bayesian Lasso regression
reg.blas <- blasso(x, y)
```

```
## t=100, m=7
## t=200, m=8
## t=300, m=6
## t=400, m=7
## t=500, m=5
## t=600, m=7
## t=700, m=9
## t=800, m=8
## t=900, m=10
```

Visualize $\hat{\beta}$

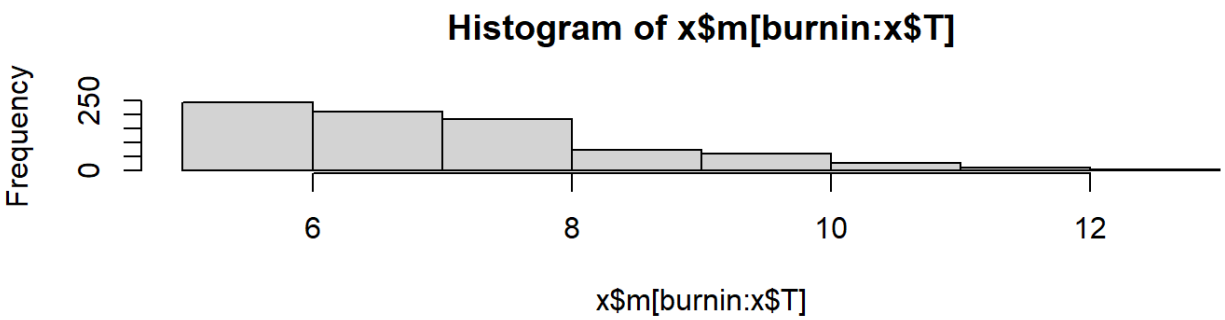
```
plot(reg.blas, burnin=200)
points(drop(reg.las$b), col=3, pch=20)
legend("topright", c("blasso-map", "lasso"), col=c(2,3), pch=c(20,18
```

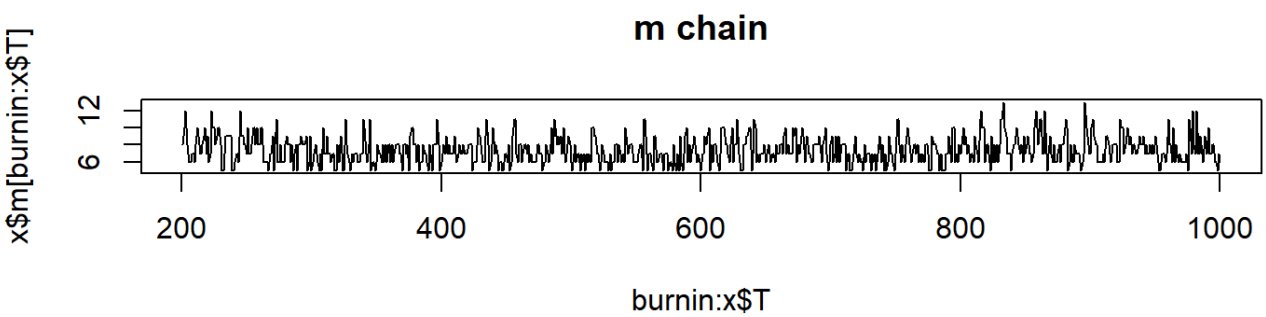


Model Dimensions

We can plot the size of different models visited:

```
plot(reg.blas, burnin=200, which="m")
```





Get the summary

The summary contains useful information about the fit.

```
s <- summary(reg.blas, burnin=200)
names(s)
```

```
##  [1] "call"    "B"       "T"       "thin"    "coef"    "s2"
##  [8] "tau2i"   "bn0"     "m"
```

Calculate $P(\beta_i \neq 0 \mid y_i)$

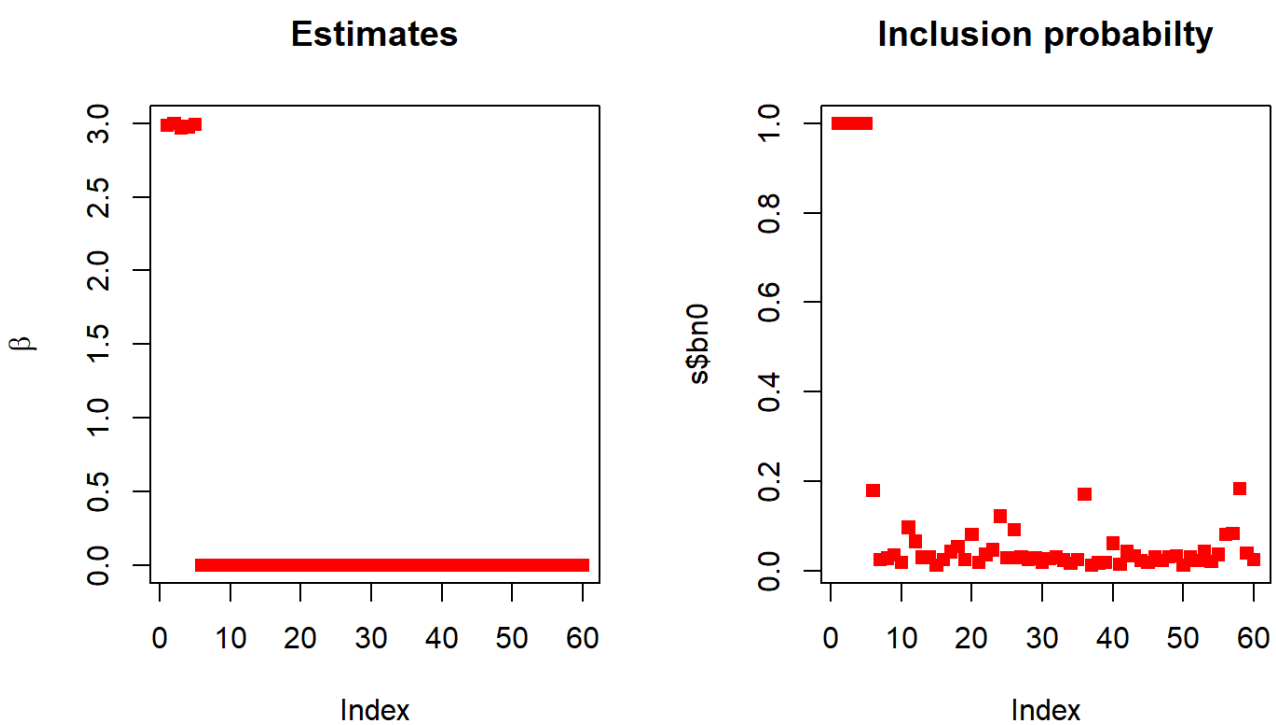
This is called posterior inclusion probability.

```
s$bn0
```

```
##     b.1     b.2     b.3     b.4     b.5     b.6     b.7     b.8
## 1.00000 1.00000 1.00000 1.00000 1.00000 0.17875 0.02500 0.02750 (
##    b.11    b.12    b.13    b.14    b.15    b.16    b.17    b.18
## 0.09625 0.06500 0.03000 0.03125 0.01250 0.02500 0.04250 0.05375 (
##    b.21    b.22    b.23    b.24    b.25    b.26    b.27    b.28
## 0.01875 0.03625 0.04625 0.12125 0.02875 0.09125 0.03000 0.02500 (
##    b.31    b.32    b.33    b.34    b.35    b.36    b.37    b.38
## 0.02625 0.03000 0.02375 0.01625 0.02500 0.17125 0.01250 0.01750 (
##    b.41    b.42    b.43    b.44    b.45    b.46    b.47    b.48
## 0.01375 0.04250 0.03250 0.02250 0.01875 0.03125 0.02250 0.03125 (
##    b.51    b.52    b.53    b.54    b.55    b.56    b.57    b.58
## 0.03125 0.02250 0.04375 0.02000 0.03625 0.08125 0.08250 0.18375 (
```

Inclusion Prob and Estimates

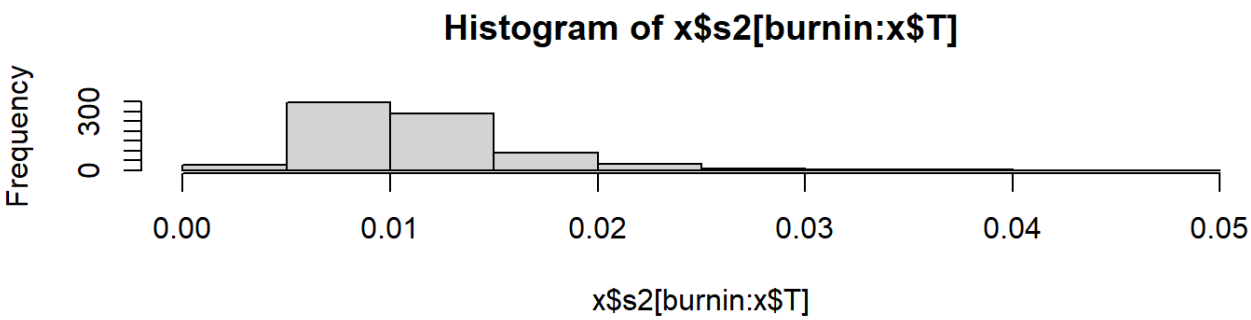
```
par(mfrow=c(1,2))
plot(apply(reg.blas$beta,2,median),pch=15,col="red", ylab = expressi
plot(s$bn0,pch=15,col="red",main = "Inclusion probabilty")
```

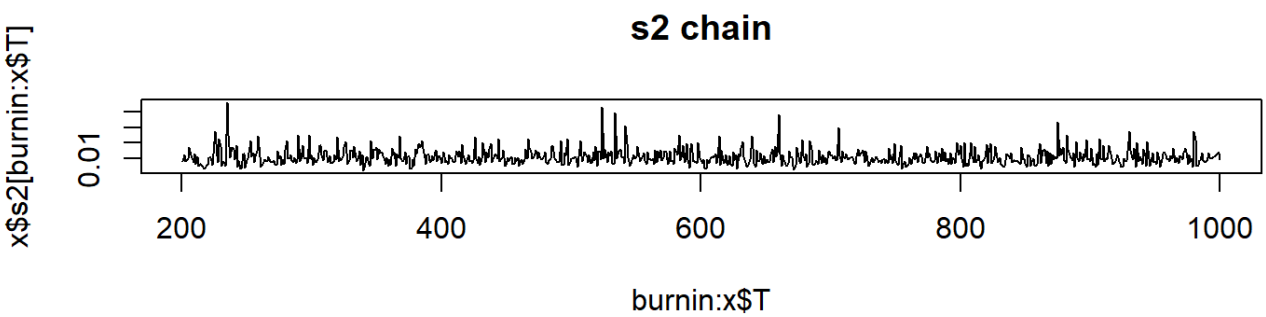


Summarize s2

`s2` is the estimates of the variance parameter.

```
plot(reg.blas, burnin=200, which="s2")
```



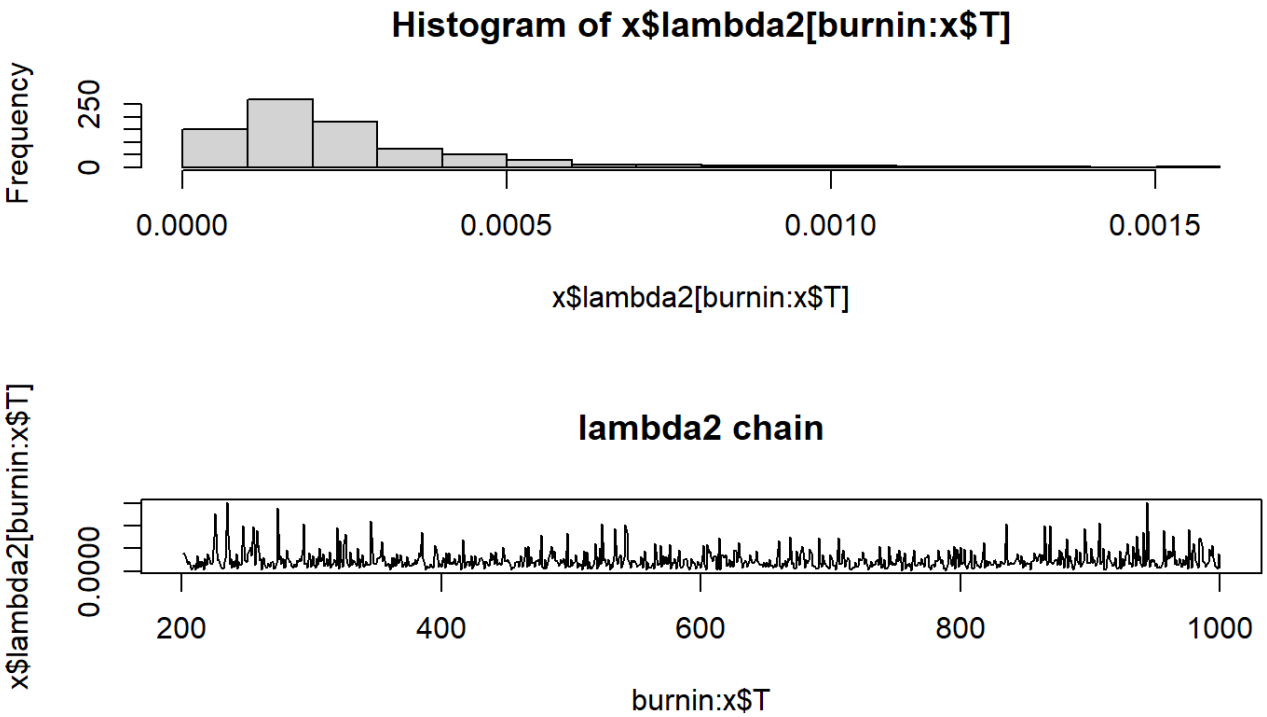



```
s$s2
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## 0.002604 0.007799 0.010321 0.011233 0.013492 0.045940
```

Summarize lambda2

```
plot(reg.blas, burnin=200, which="lambda2")
```




```
s$lambda2
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## 2.096e-05 1.177e-04 1.936e-04 2.506e-04 3.017e-04 1.515e-03
```

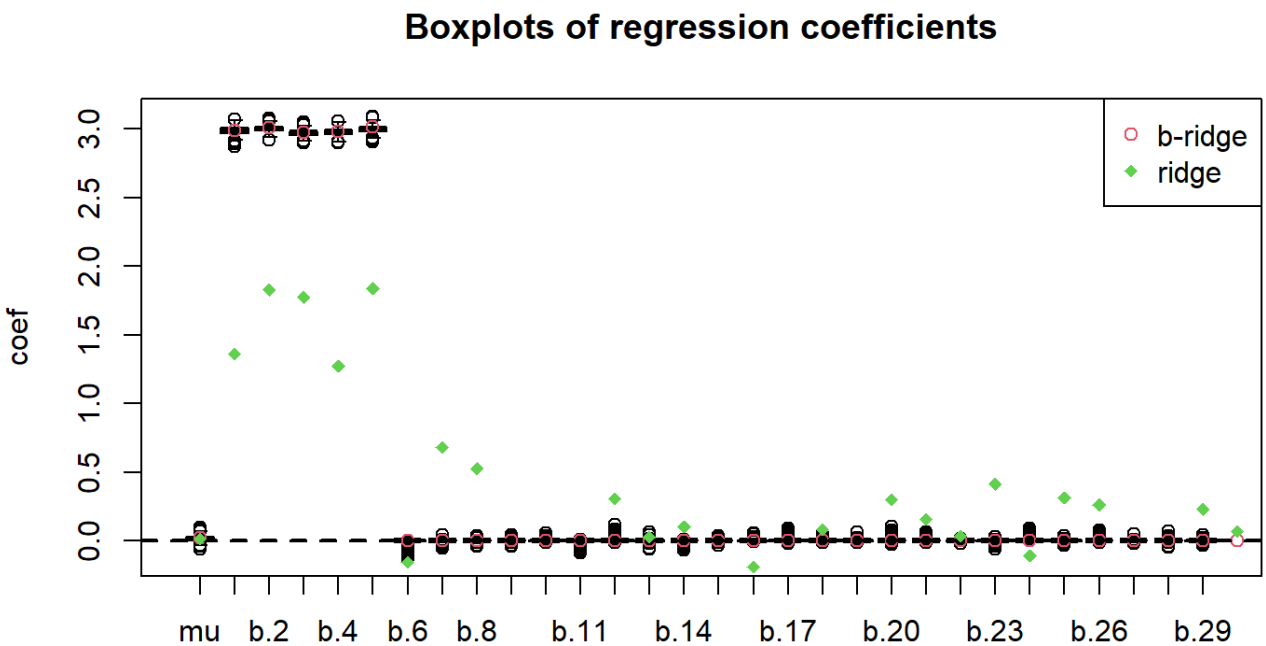
Bayesian Ridge

```
## Ridge regression
reg.ridge <- regress(x, y, method="ridge")
## Bayesian Ridge
reg.bridge <- bridge(x, y)
```

```
## t=100, m=5
## t=200, m=6
## t=300, m=6
## t=400, m=5
## t=500, m=9
## t=600, m=8
## t=700, m=6
## t=800, m=6
## t=900, m=8
```

Visualize $\hat{\beta}$

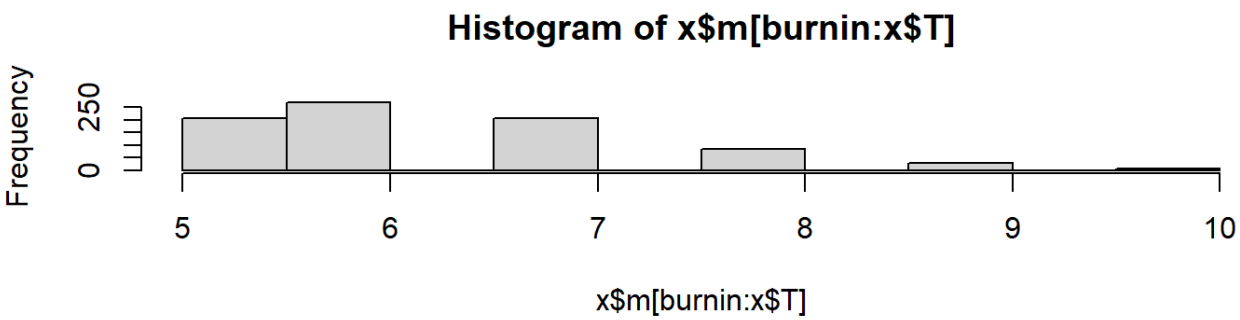
```
plot(reg.bridge, burnin=200)
points(drop(reg.ridge$b), col=3, pch=18)
legend("topright", c("b-ridge", "ridge"), col=c(2,3), pch=c(21,18))
```

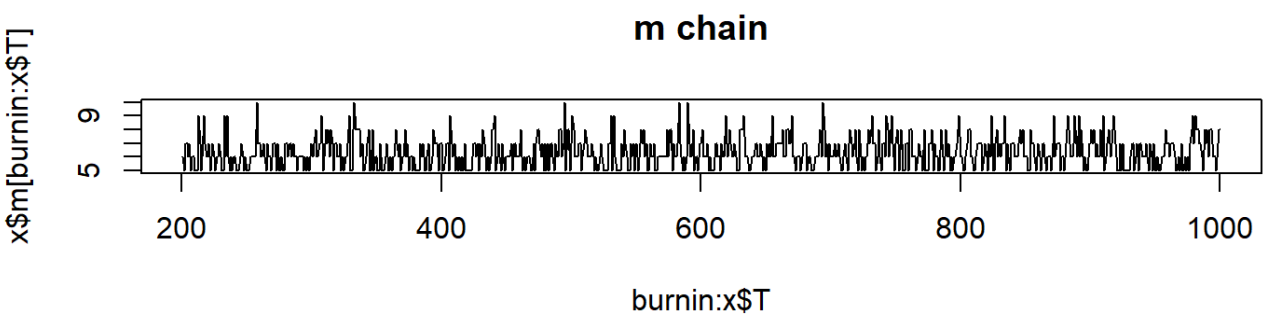


Model Dimensions

We can plot the size of different models visited:

```
plot(reg.bridge, burnin=200, which="m")
```



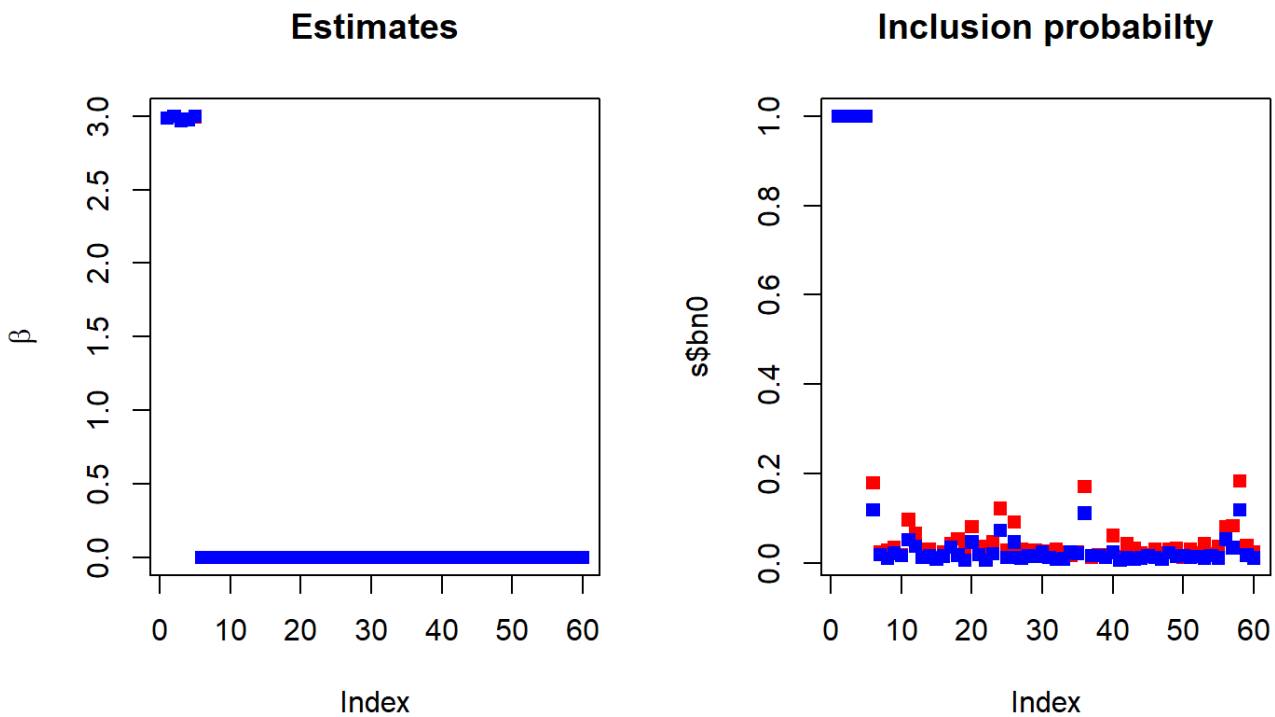


Calculate $P(\beta_i \neq 0 \mid y_i)$

This is called posterior inclusion probability.

```
sridge <- summary(reg.bridge, burnin=200)
sridge$bn0
```

```
##     b.1     b.2     b.3     b.4     b.5     b.6     b.7     b.8
## 1.00000 1.00000 1.00000 1.00000 1.00000 0.11875 0.01875 0.01000 (
##    b.11    b.12    b.13    b.14    b.15    b.16    b.17    b.18
## 0.05125 0.03750 0.01250 0.01500 0.00875 0.01500 0.03500 0.01750 (
##    b.21    b.22    b.23    b.24    b.25    b.26    b.27    b.28
## 0.01875 0.00625 0.02000 0.07250 0.01250 0.04625 0.01125 0.01500 (
##    b.31    b.32    b.33    b.34    b.35    b.36    b.37    b.38
## 0.01250 0.00750 0.00875 0.02375 0.02125 0.11125 0.01625 0.01625 (
##    b.41    b.42    b.43    b.44    b.45    b.46    b.47    b.48
## 0.00625 0.01250 0.00750 0.01125 0.01500 0.01250 0.00750 0.02250 (
##    b.51    b.52    b.53    b.54    b.55    b.56    b.57    b.58
## 0.01250 0.01375 0.01000 0.01625 0.01125 0.05250 0.03375 0.11875 (
```

Compare

```
par(mfrow=c(1,2))
plot(apply(reg.blas$beta,2,median),pch=15,col="red", ylab = expressi
points(apply(reg.bridge$beta,2,median),pch=15,col="blue")
plot(s$bn0,pch=15,col="red",main = "Inclusion probabilty")
points(sridge$bn0,pch=15,col="blue")
```